



SDA WEEK 12 (LEC 1)

IMPLEMENTATION DIAGRAMS

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

INTRODUCTION

So far, we've covered:

- Static diagrams:
 - Use Case
 - Class diagrams
- Dynamic diagrams:
 - Sequence diagrams
 - Activity diagrams
 - State diagrams
 - Collaboration diagrams

INTRODUCTION

Now, we shift focus toward the Implementation View, which focuses on how the system is physically built, the actual building blocks of software.

The two important UML diagrams in this category are:

- 1. Component Diagram**
- 2. Deployment Diagram**

COMPONENT DIAGRAM

- In UML, a component is an autonomous, self-contained unit within a system.
- Think of it as a black box that provides certain services and hides its internal details.

Example (Imagine your smartphone):

- It has a Camera Component, a Messaging Component, a Bluetooth Component, and so on. Each one performs its function without exposing internal complexity.

COMPONENT DIAGRAM

In software, a component could be:

- A source code file (like login.java)
- A library (.dll or .jar)
- A database table
- Even an executable file (.exe)

Each component encapsulates part of the system's functionality and can be reused or replaced independently, provided the interfaces match.

COMPONENT DIAGRAM

- A Component Diagram helps us visualize how a system is divided into logical pieces. Showing components, their interfaces, and how they interact.
- It focuses on the physical implementation of the system rather than the conceptual design.
- In short, it answers the question:
 - How is my software system implemented in terms of components and their connections?

COMPONENT DIAGRAM

- **Key Characteristics:**
 - It models high-level software components and their interfaces.
 - It shows dependencies between components.
 - It focuses on independent deployable units.

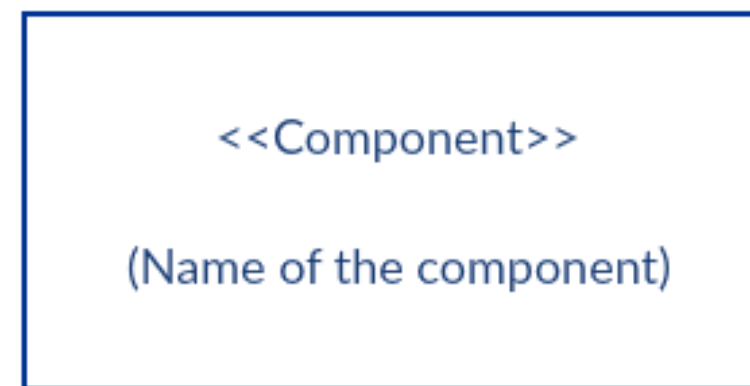
ELEMENTS OF COMPONENT DIAGRAM

Elements	Description
Component	The main building block, shown as a rectangle with <<component>> stereotype
Interface	Defines how components interact, the operations a component provides or requires
Port	A connection point for communication
Connector	The link showing interaction between components (either assembly or delegation)

COMPONENT

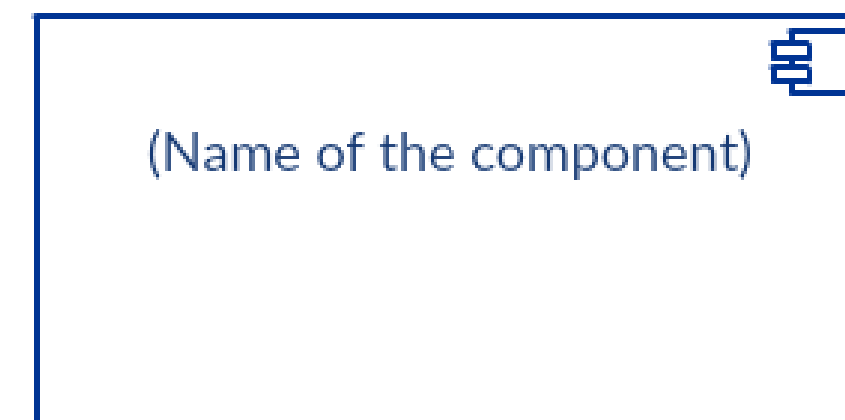
Notation 1:

Rectangle with the component stereotype (the text `<<component>>`). The component stereotype is usually used above the component name to avoid confusing the shape with a class icon.



Notation 2:

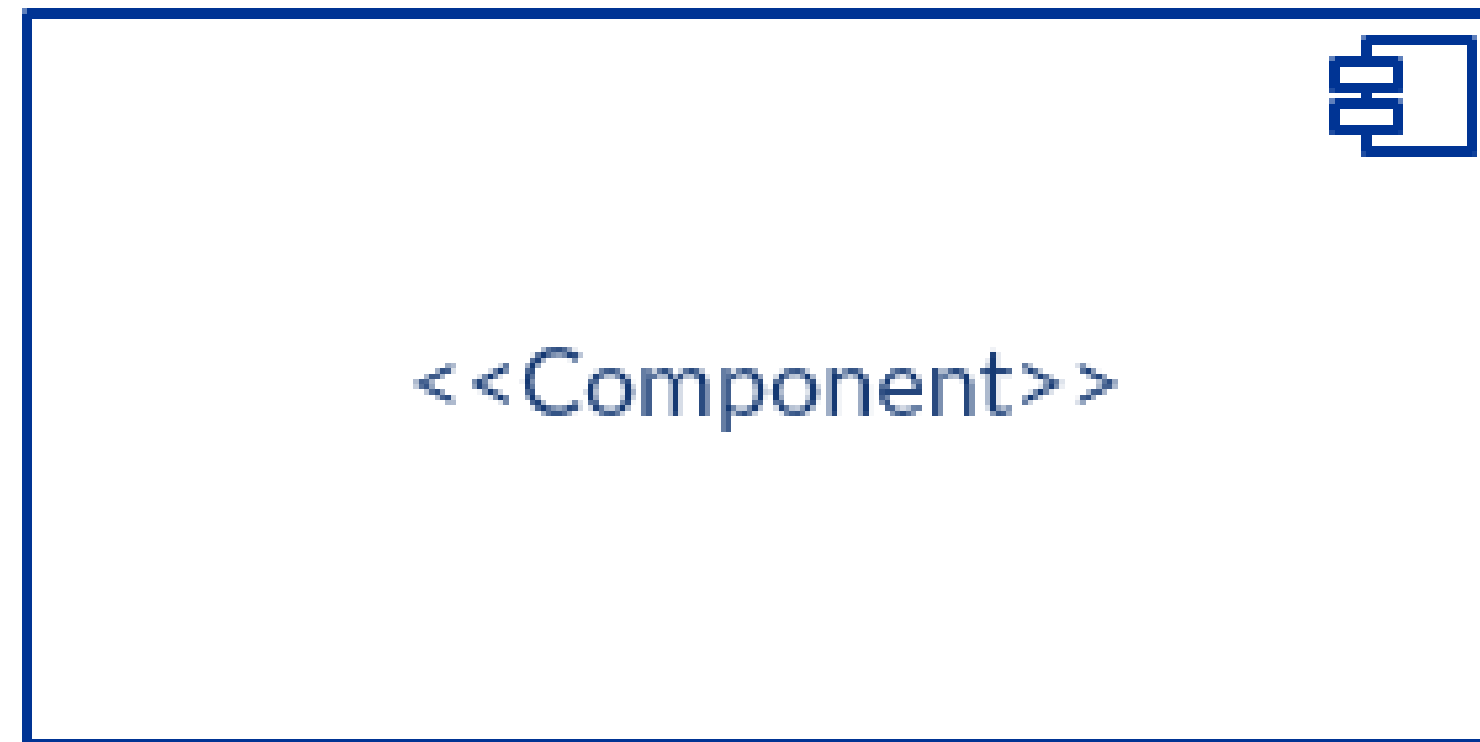
Rectangle with the component icon in the top right corner and the name of the component.



COMPONENT

Notation 3:

A rectangle with both the component icon and the stereotype label.



COMPONENT

«component»

Order



Order

«component»

Order



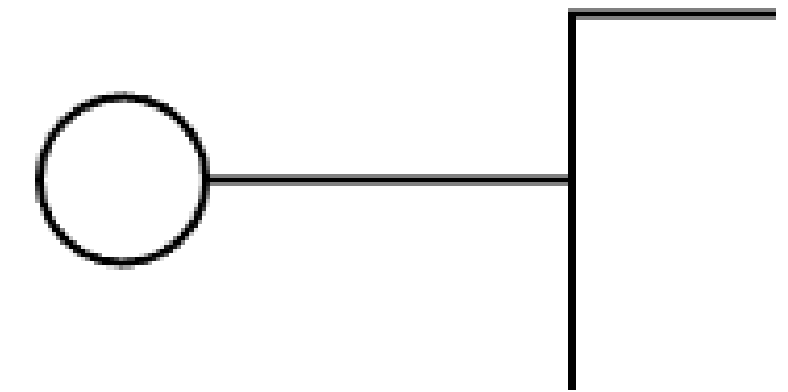
COMPONENT STEREOTYPES

Stereotype	Meaning
<<application>>	A front-end application (e.g., GUI or web pages)
<<database>>	Represents a database or data source
<<library>>	Function or object library
<<executable>>	Executable file
<<table>>	A table in a database
<<web service>>	A web-based service interface

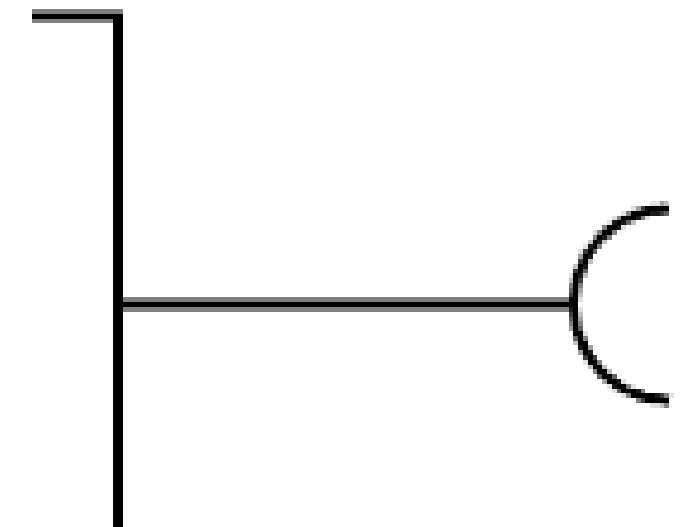
INTERFACES

Interfaces define how components talk to each other, what services one component offers and what services it expects.

Provided interfaces: A straight line from the component box with an attached circle. These symbols represent the interfaces where a component produces information used by the required interface of another component.

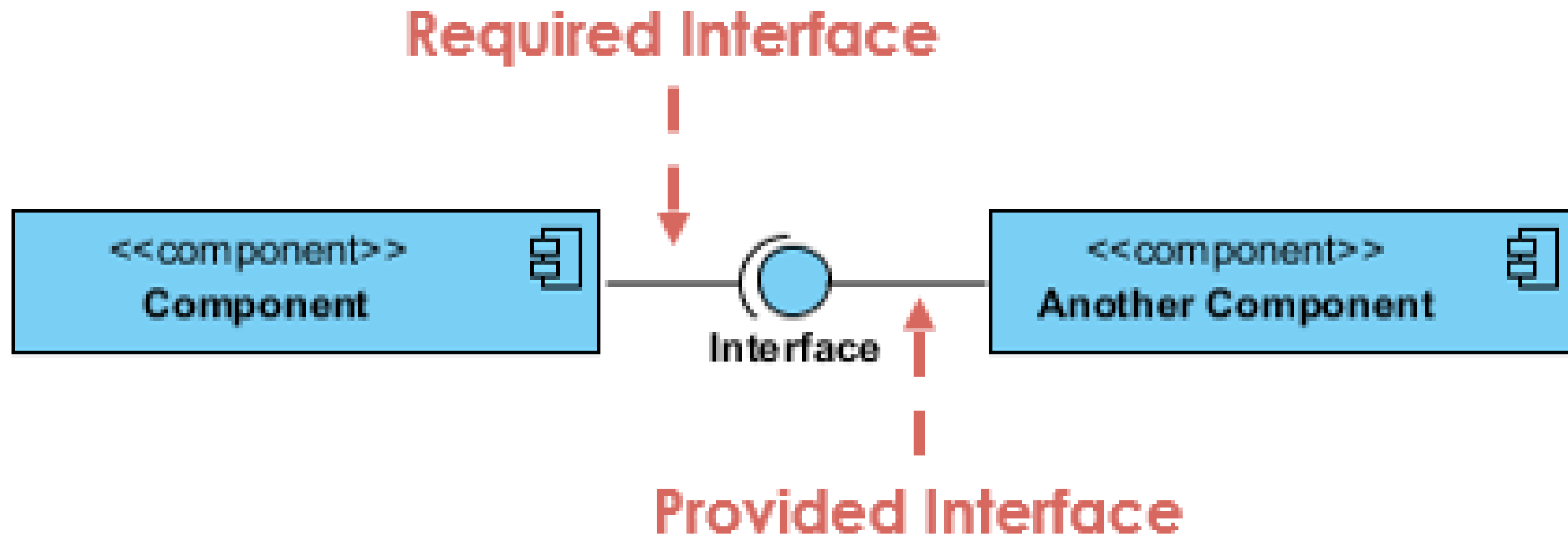


Required interfaces: A straight line from the component box with an attached half circle (also represented as a dashed arrow with an open arrow). These symbols represent the interfaces where a component requires information in order to perform its proper function.



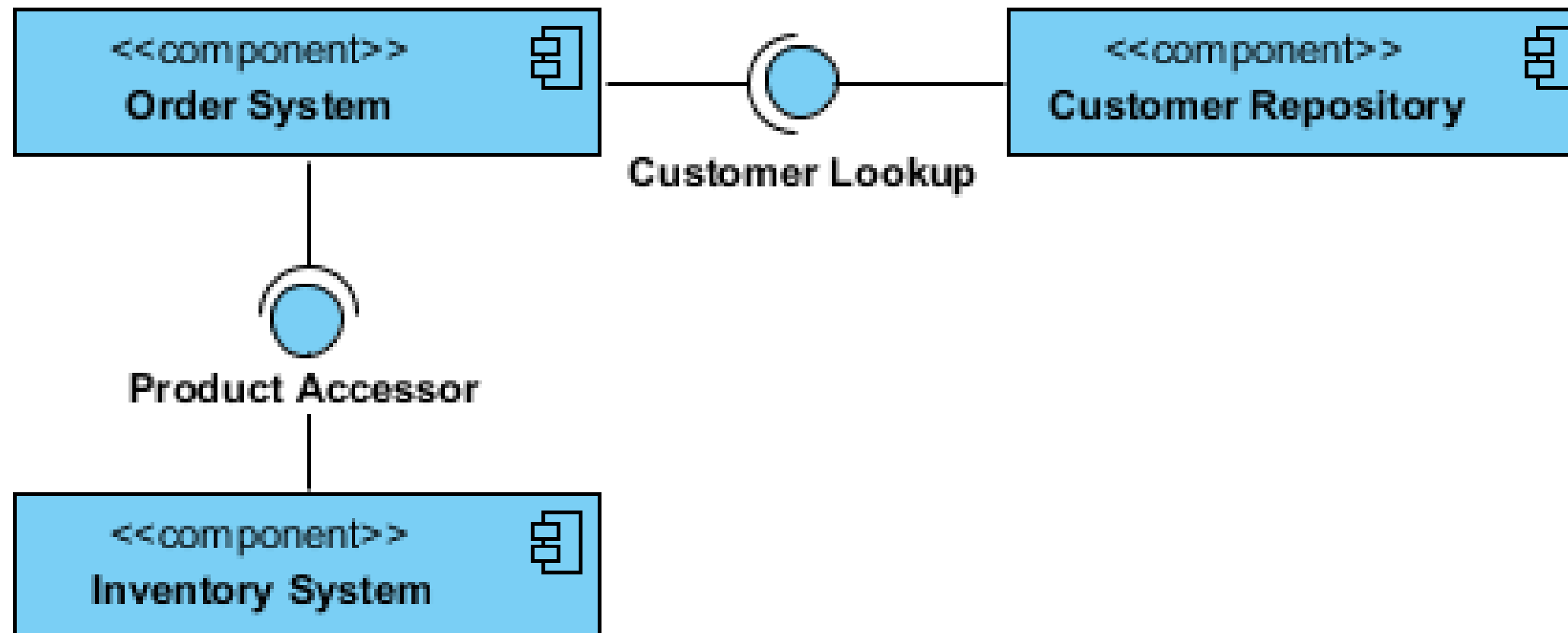
INTERFACES

Interfaces define how components talk to each other, what services one component offers and what services it expects.



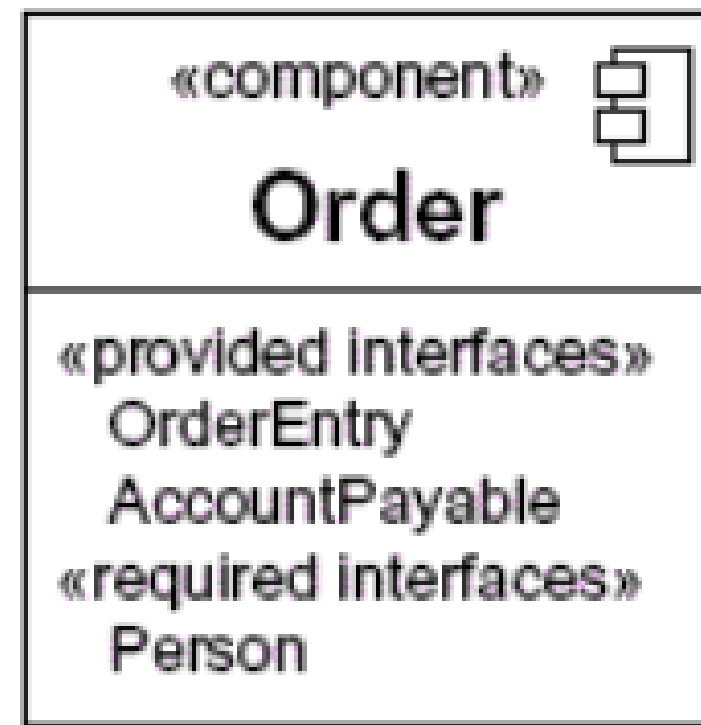
INTERFACES

Component Diagram Example - Using Interface (Order System)



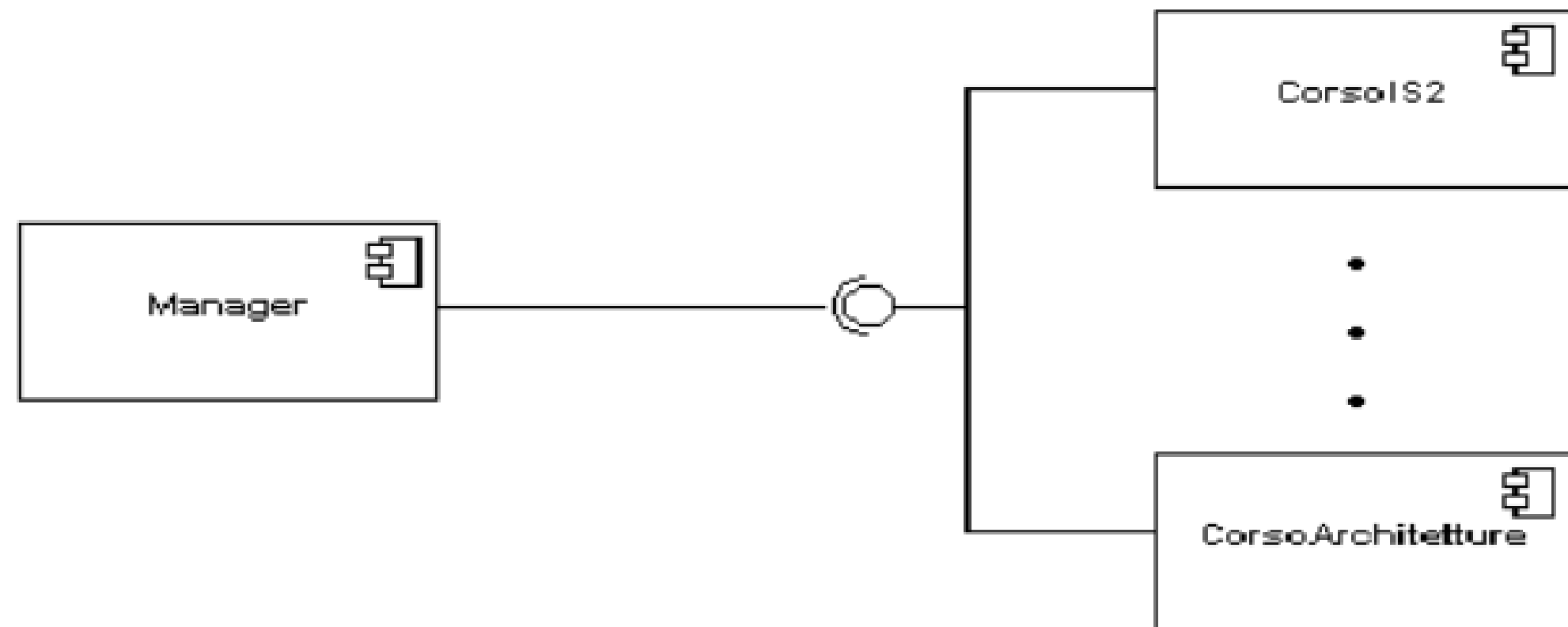
INTERFACES

As with the class notation, components also have an optional space to list interfaces, similar to the way you add attributes and methods to class notation. Interfaces represent the places where the groups of classes in the component communicate with other system components. An alternative way to represent interfaces is by extending symbols from the component box.



INTERFACES

In a system context where there are multiple components that require or provide a particular interface, a notation abstraction can be used that combines by joining the interfaces.



CONNECTORS

There are two types of connectors:

1. **Assembly Connector:**

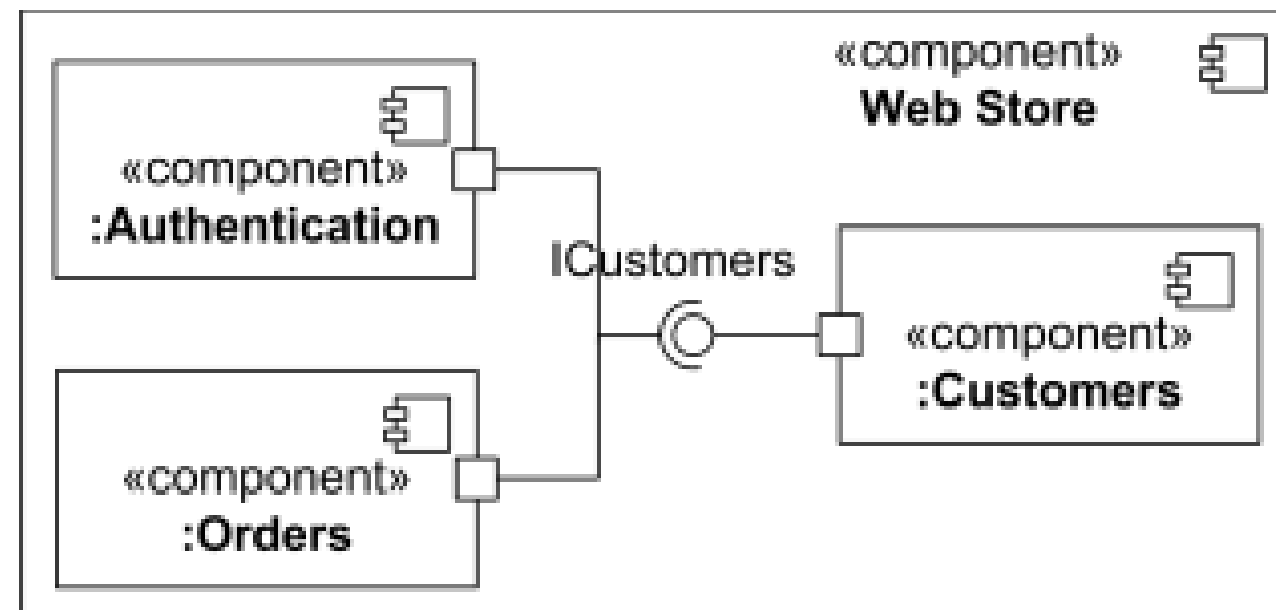
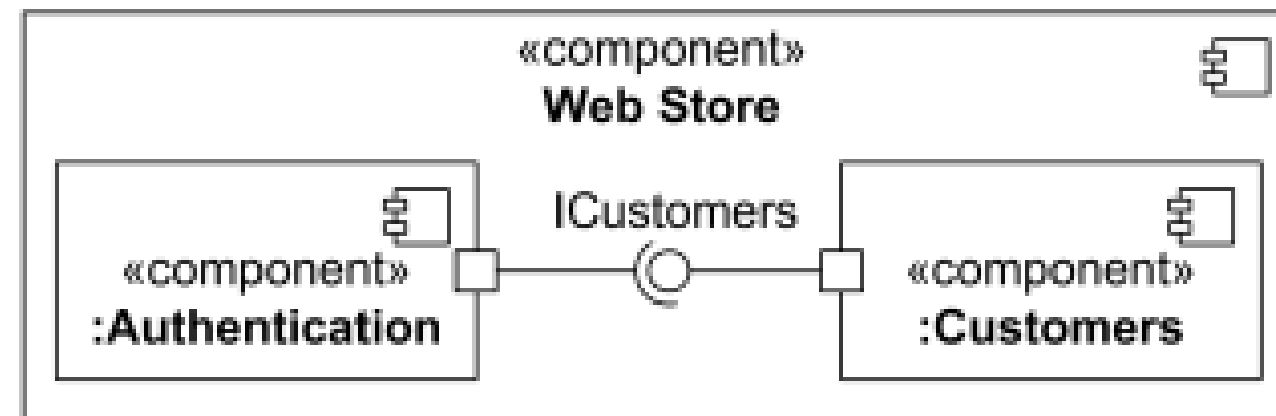
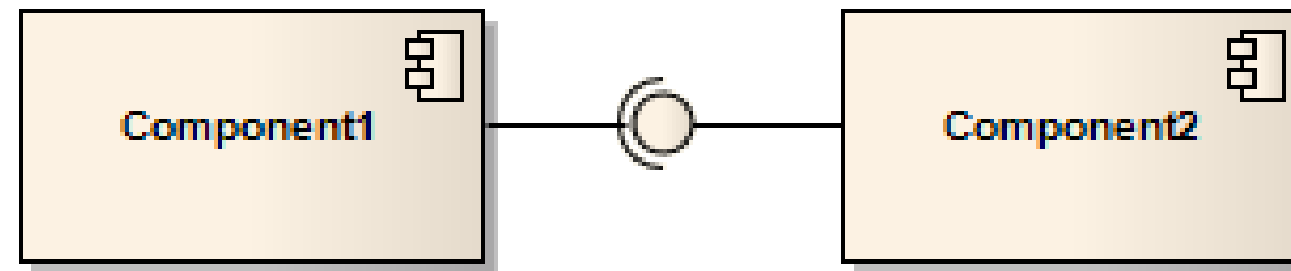
- Connects a required interface of one component to a provided interface of another.
- Think of it as “plugging” one component into another.

2. **Delegation Connector:**

- Used when a component passes responsibility to an internal component.
- Shows that internal parts realize or use the external interfaces.

ASSEMBLY CONNECTORS

Assembly connector
between simple ports of
Authentication and
Customers components.



Assembly connector that
assembles three parts.



DELEGATION CONNECTORS

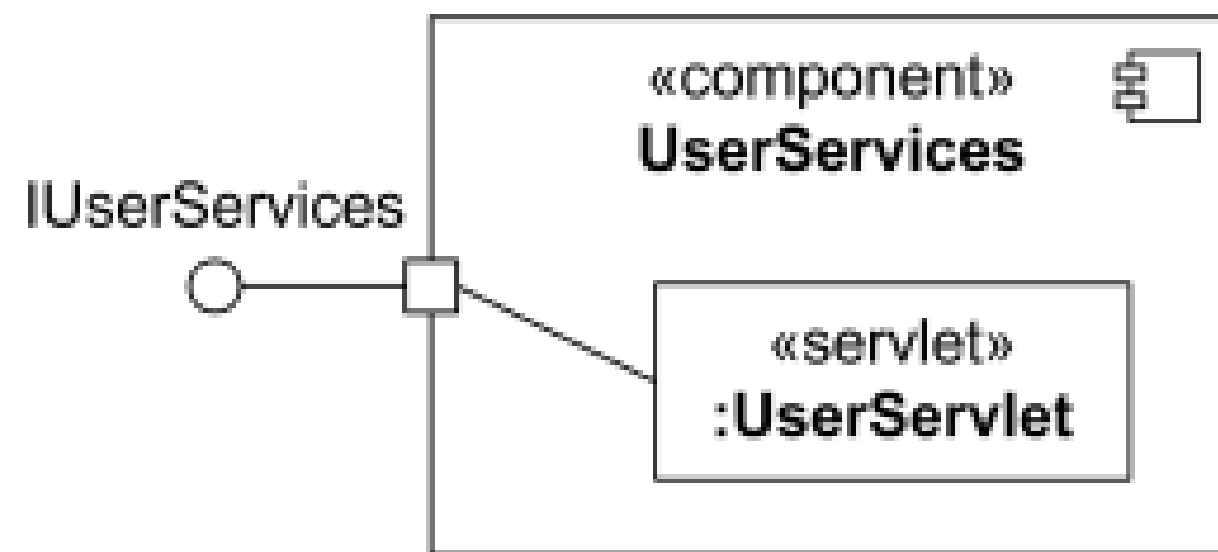
A delegation connector is a connector that links the external contract of a component (as specified by its ports) to the realization of that behavior. It represents the forwarding of events (operation requests and events): a signal that arrives at a port that has a delegation connector to one or more parts or ports on parts will be passed on to those targets for handling.

A delegation connector is notated as a connector from the delegating port to the handling port or part.

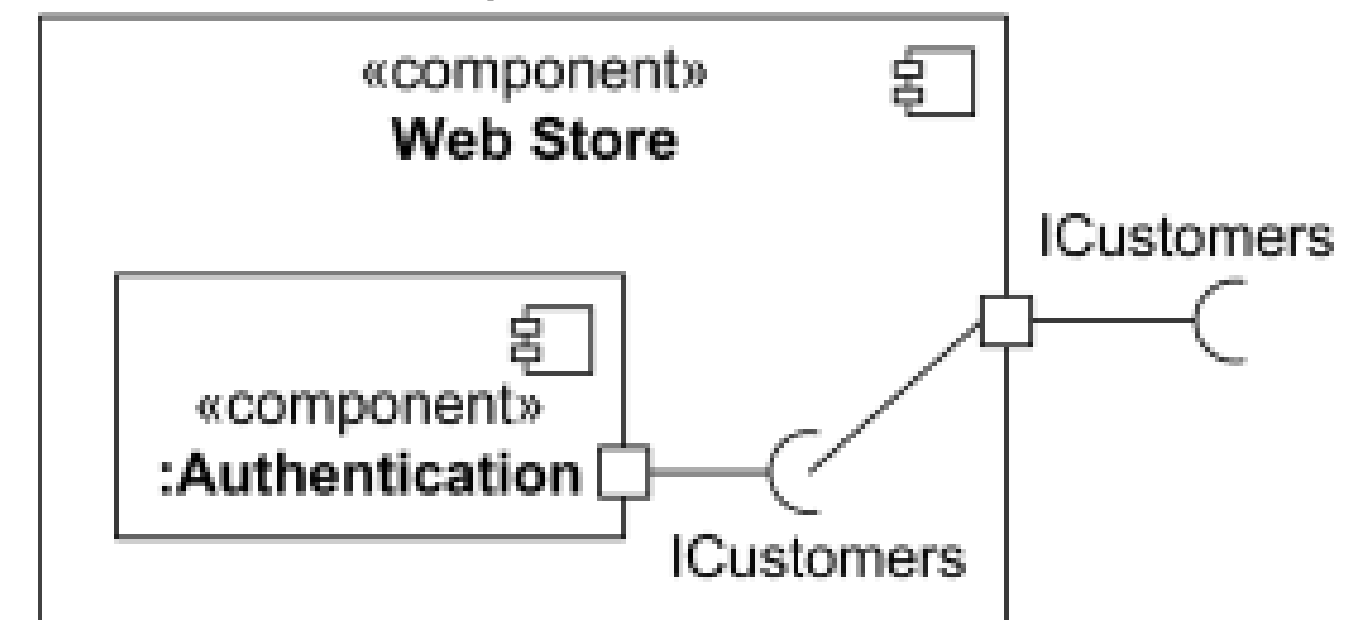


DELEGATION CONNECTORS

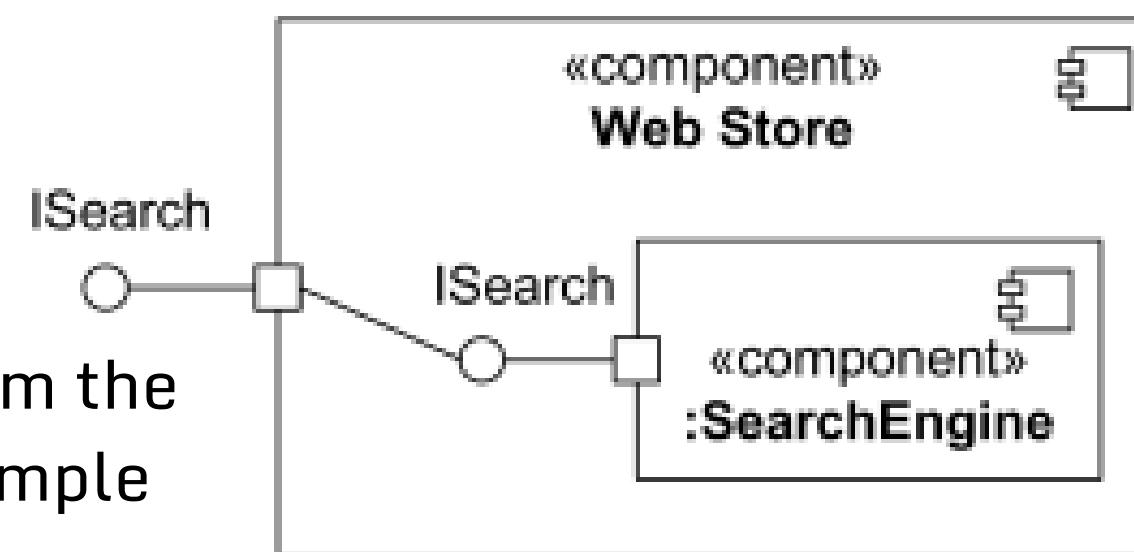
Delegation connector from the delegating port to the UserServicelet part



Delegation connector from the simple port of Authentication component to the delegating port.

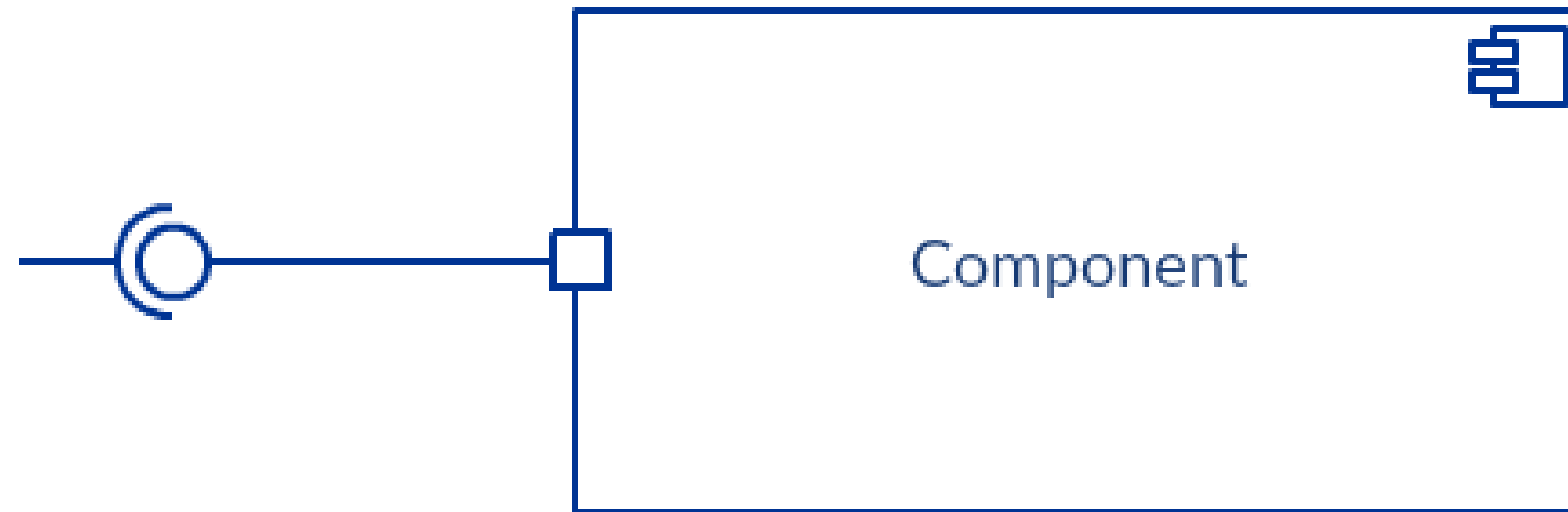


Delegation connector from the delegating port to the simple port of SearchEngine.



PORTS

A port is a small square on the edge of a component, used to delegate interfaces to internal classes or subcomponents. Ports help organize interactions, especially in large or complex systems, and clarify which component handles which interface.



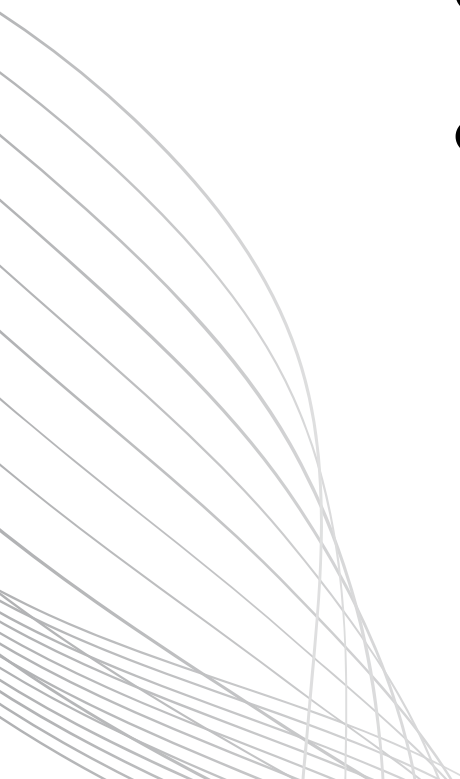


PORTS

A port defines a specific interaction point between a component and its environment.

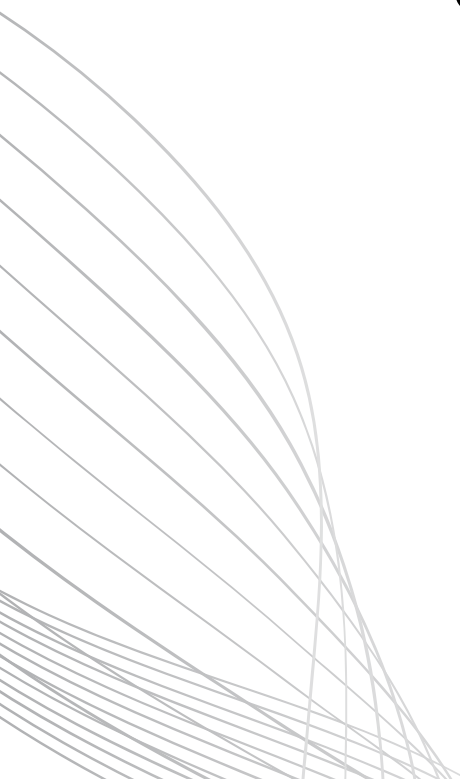
It's drawn as a small square on the component boundary.

Ports:

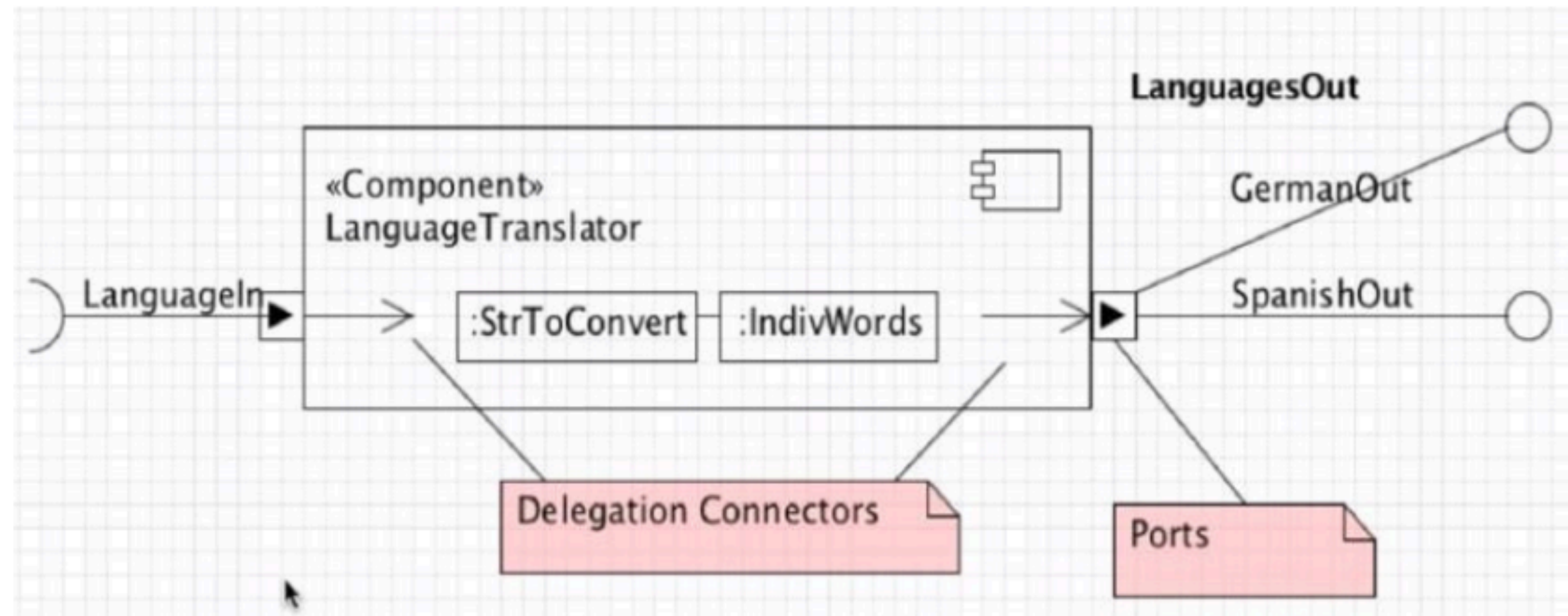
- Can handle uni- or bi-directional communication
 - Can be named (e.g., searchPort)
 - Can be linked with one or multiple interfaces
- 



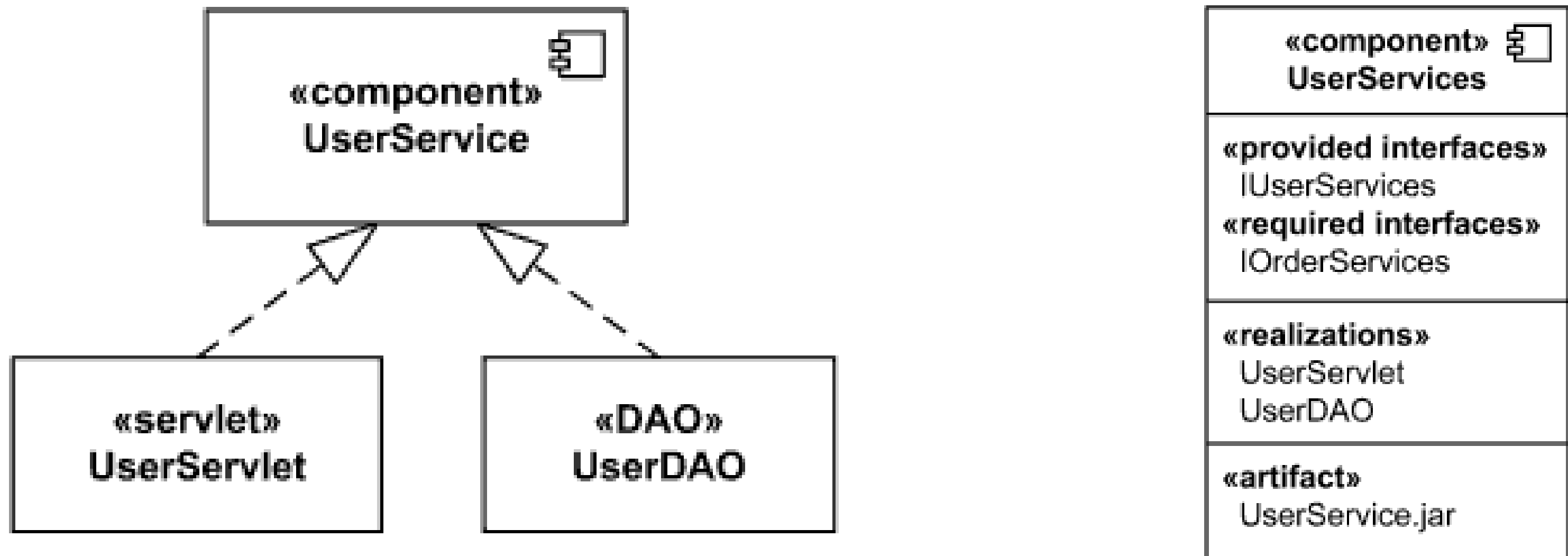
INTERNAL REALIZATION

- Components may also contain internal realizing classes that implement their behavior. These are shown inside the component box, the white-box view.
 - This simply means that the behavior promised by the component's external interfaces is physically implemented by the classes, components, or artifacts inside it.
 - The Delegation Connectors are the visual proof of this realization. They show which internal part is responsible for which external promise.
- 

INTERNAL REALIZATION



INTERNAL REALIZATION



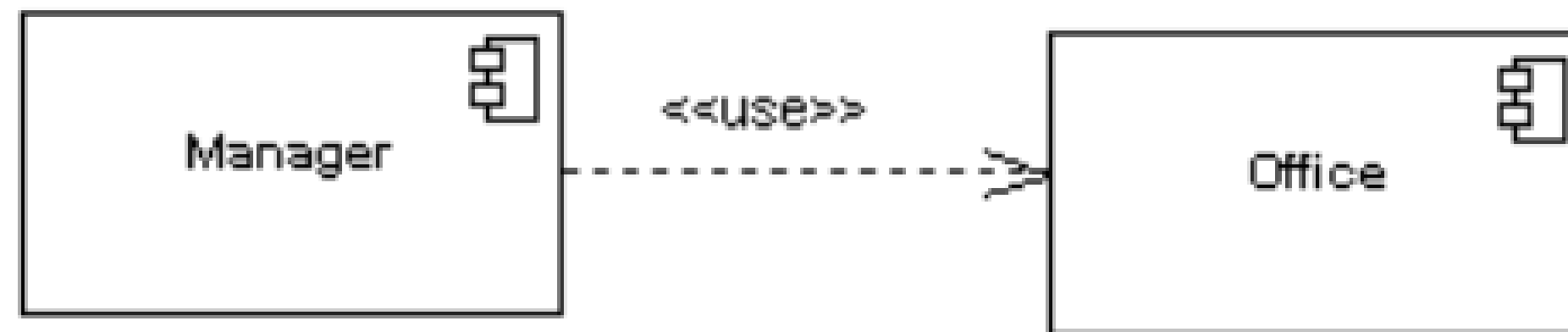
DEPENDENCY

- A relationship in which one element requires another element for its full implementation
 - This means the dependent component (client) cannot function completely without the supplier component
- A dependency in which the client requires the presence of the supplier
- The client component needs the supplier component to be present in the system If the supplier is missing, the client won't work properly
 - Shown as dashed arrow with a <<use>> keyword

DEPENDENCY

- **Key Properties:**

- **Directional:** The dependency has a clear direction—from the client (who needs something) to the supplier (who provides it).
- **Not a Direct Connection:** Unlike an assembly connector (ball-and-socket), a usage dependency does not specify the exact service or interface being used. It simply states that a dependency exists.



COMPONENT VIEWS

There are two main views of a component:

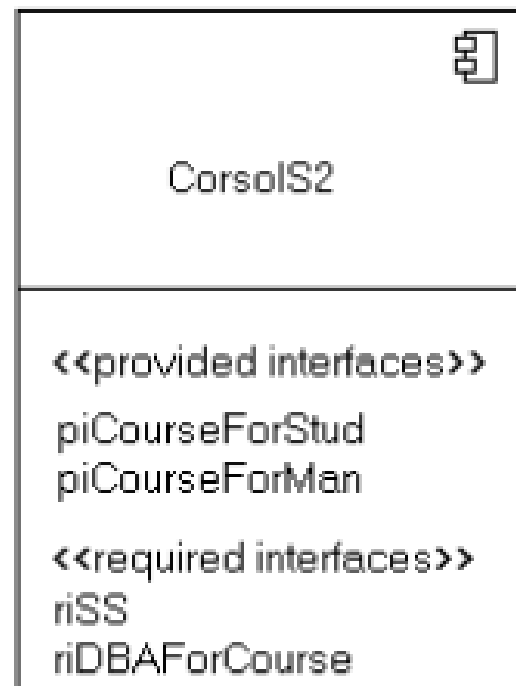
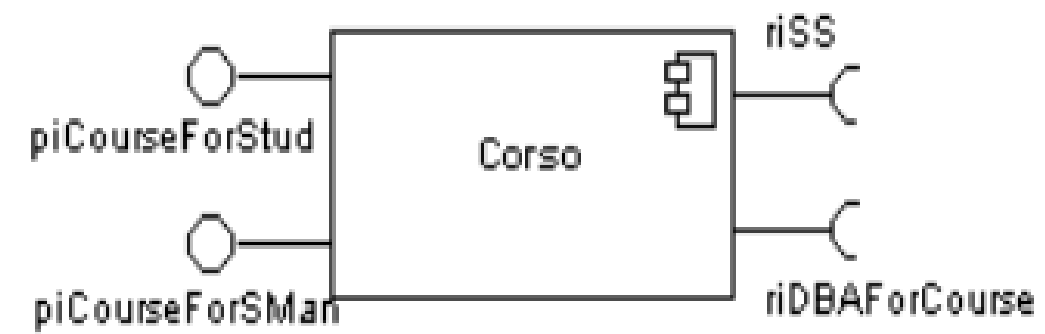
1. External (Black-Box) View:

- Shows only what is visible to the outside world – the interfaces.
- Focuses on what the component does, not how it does it.

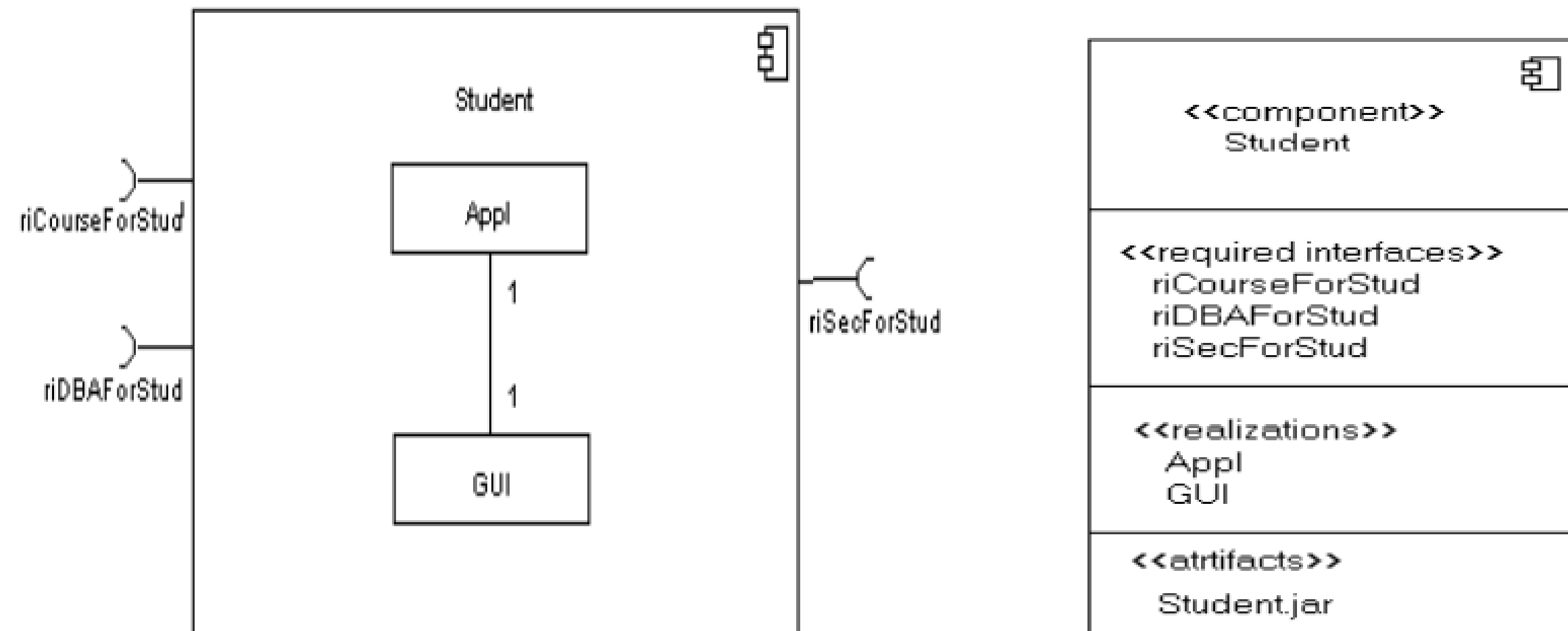
2. Internal (White-Box) View:

- Shows the internal structure, including realizing classes and internal connectors.
- Focuses on how the component achieves its behavior.

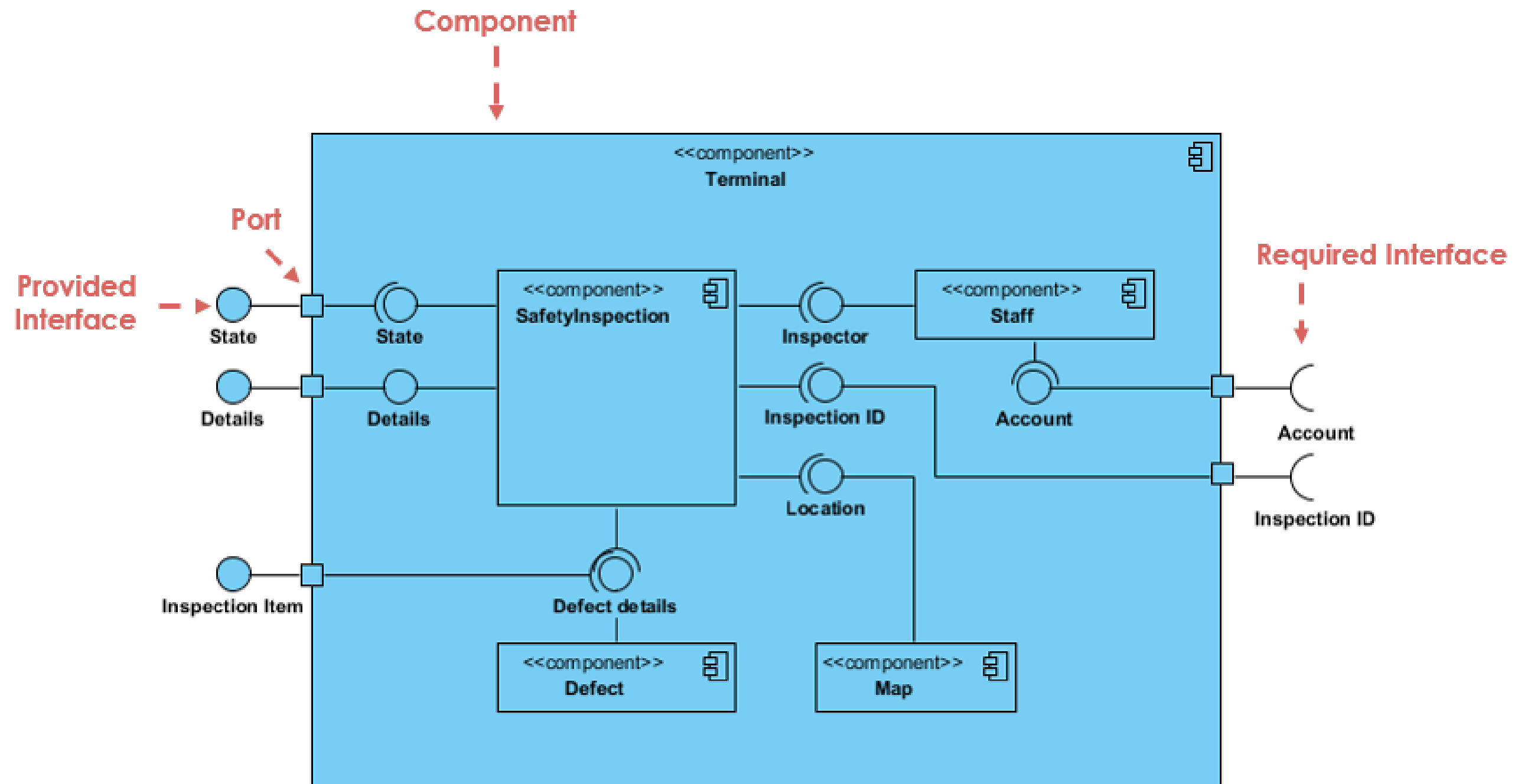
EXTERNAL VIEW



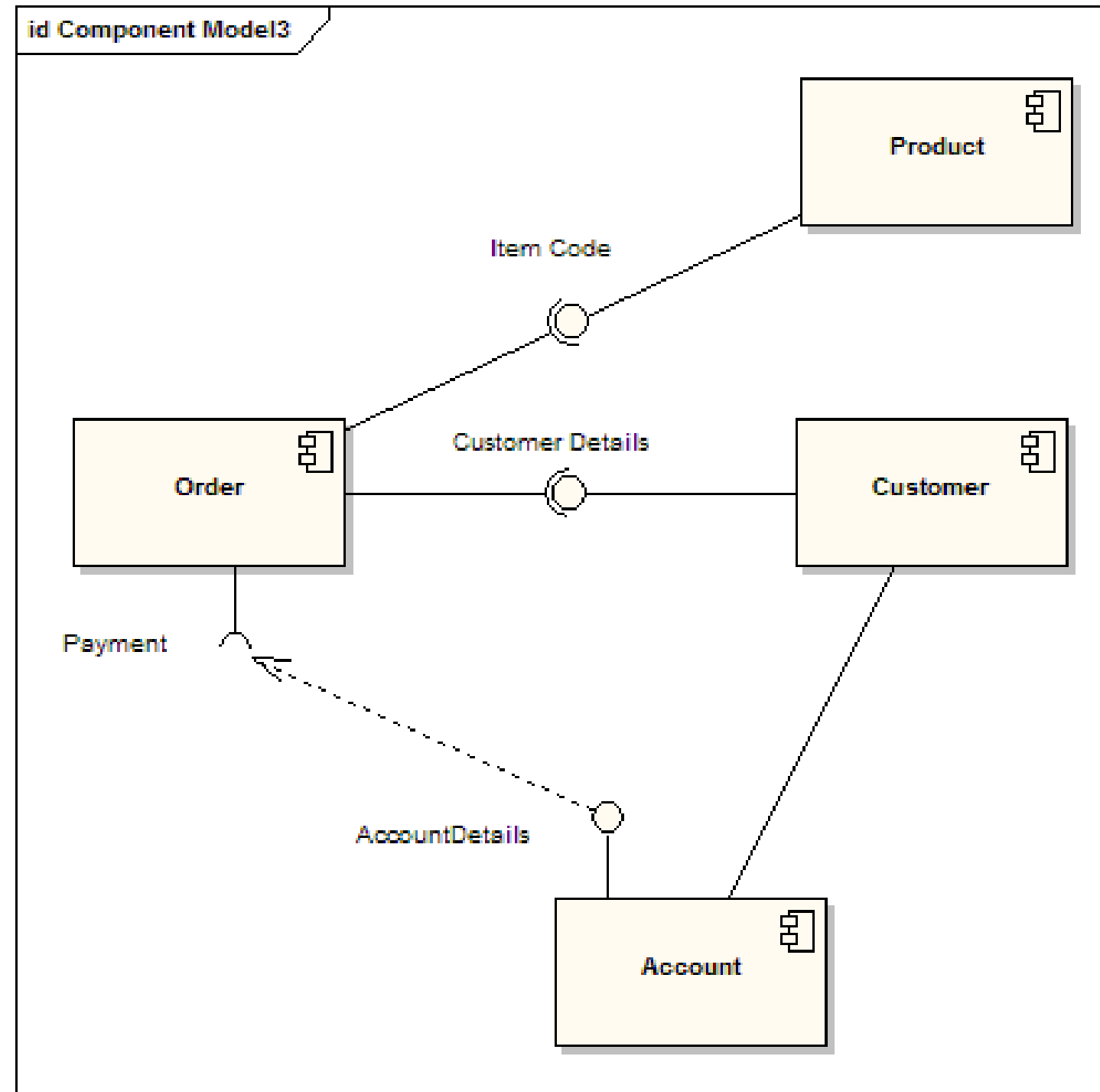
INTERNAL VIEW



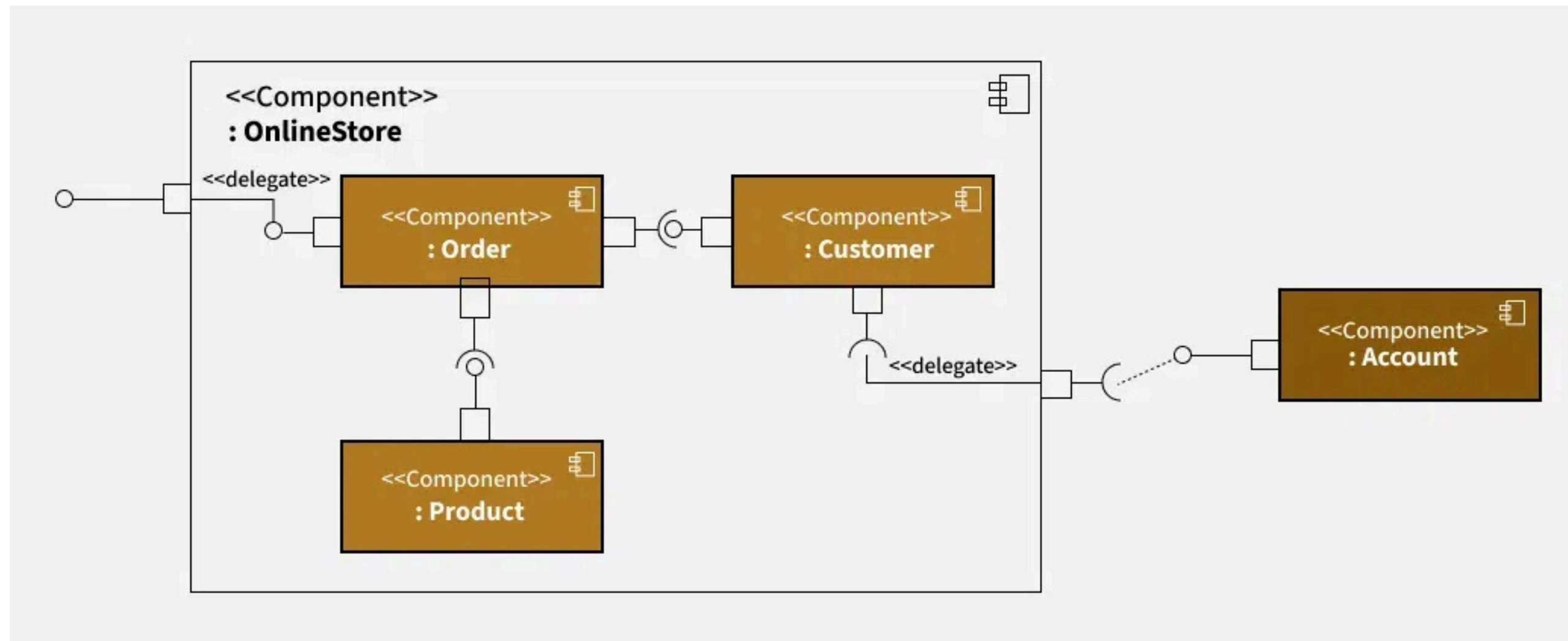
EXAMPLE



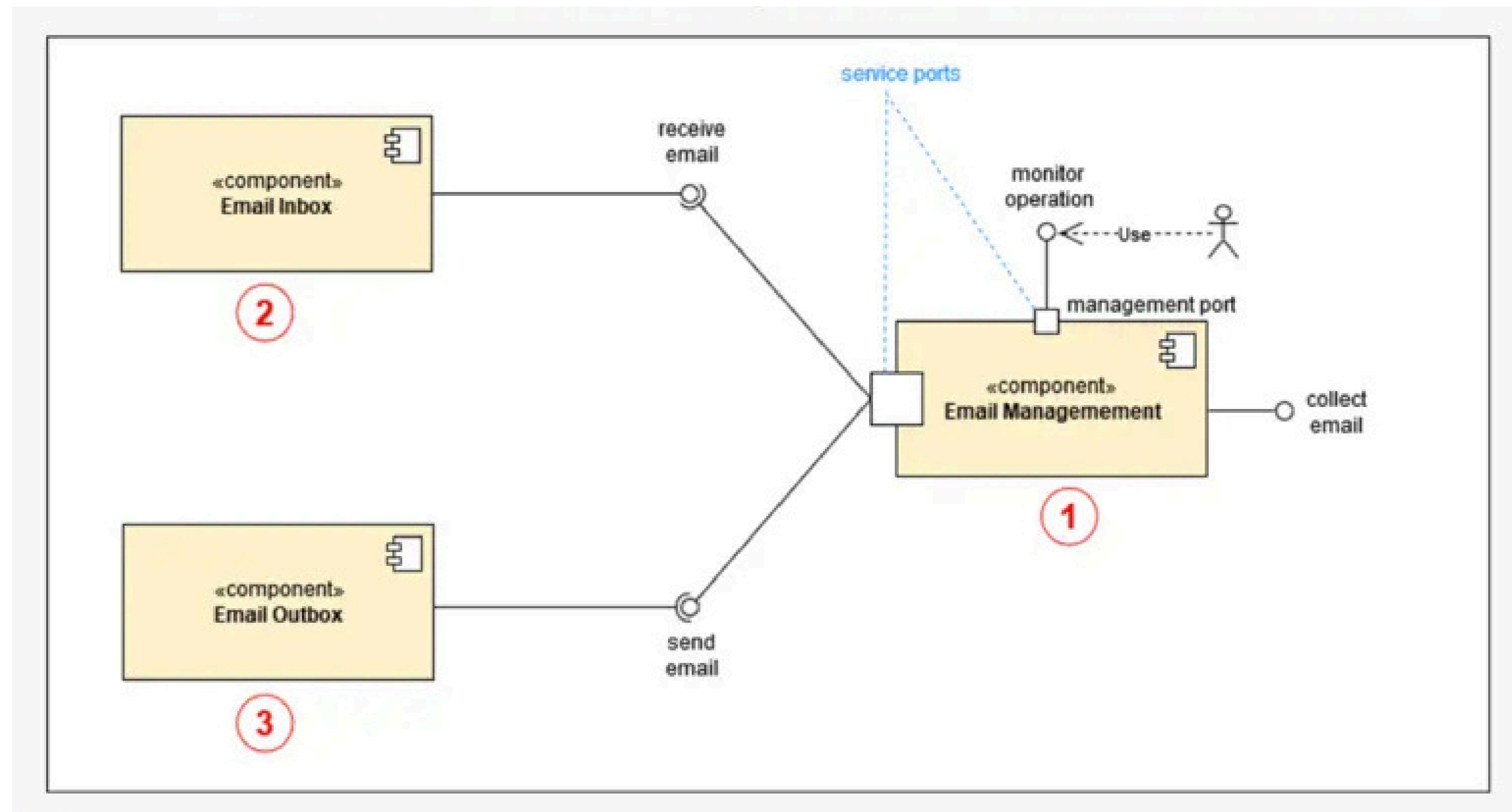
EXAMPLE



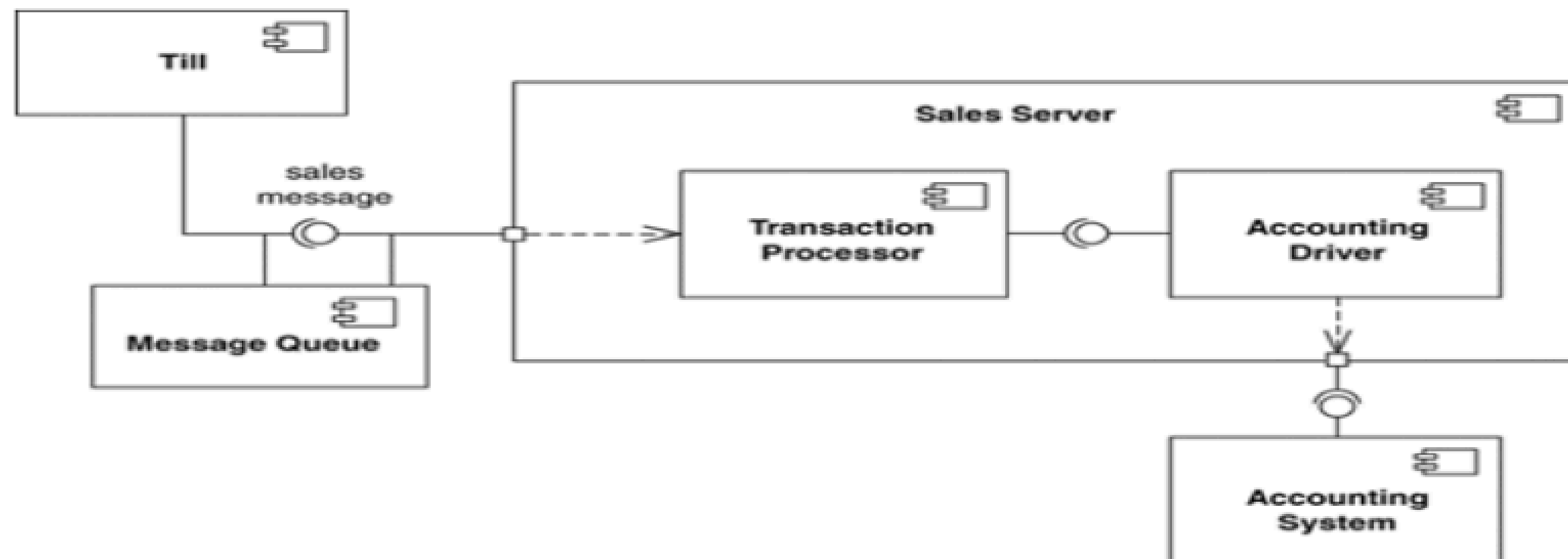
EXAMPLE: ONLINE STORE



EXAMPLE: EMAIL SYSTEM



A sales till can connect to a sales server component, using a sales message interface. Because the network is unreliable, a message queue component is set up so the till can talk to the server when the network is up and talk to a queue when the network is down; the queue will then talk to the server when the network becomes available. As a result, the message queue both supplies the sales message interface to talk with the till and requires that interface to talk with the server. The server is broken down into two major components. The transaction processor realizes the sales message interface, and the accounting driver. Accounting driver talks to the accounting system component for getting the details about the bill information. Accounting driver provides this information to transaction processor for managing the sale.





THANK YOU